《操作系统综合实验》 实验报告

实验名称	多线程与信号量编程	实验序号	4
实验日期	2024.5.31	实验人	廖鑫洋

一、 实验题目

生产者——消费者问题

在第 5.7.1 节中，我们提出了一种使用有界缓冲区的信号量解决方案来解决生产者消费者问题。你将使用图 5.9 和 5.10 中所示的生产者和消费者进程来设计有界缓冲区问题的编程解决方案。5.7.1 节中介绍的解决方案使用三个信号量：empty 和 full，它们计算缓冲区中空槽和满槽的数量，以及 mutex，它是一个二进制（或互斥）信号量，用于保护实际插入或删除缓冲区中的项目。你将使用标准计数信号量表示 empty 和 full，并使用互斥锁而不是二进制信号量来表示 mutex。生产者和消费者作为单独的线程运行——将产品移入和移出与 empty、full 和 mutex 结构同步的缓冲区。可以使用 Pthreads 或 Windows API 实现。

生产者和消费者线程

生产者线程将在随机休眠一段时间和将随机整数插入缓冲区之间交替。随机数将使用 rand() 函数产生，该函数产生介于 0 和 RAND_MAX 之间的随机整数。消费者也会随机休眠一段时间，醒来后会尝试从缓冲区中删除一个项目。生产者和消费者线程的概要如图 5.26 所示。

如前所述，您可以使用 Pthreads 或 Windows API 解决此问题。在以下部分中，我们将提供有关每个选择的更多信息。

二、 相关原理与知识

1、生产者——消费者问题

存在两个类型的线程：生产者和消费者。生产者生成数据并将其放入缓冲区，而消费者从缓冲区中取出数据并处理。如果缓冲区满了，生产

者必须等待消费者消费数据后再继续生产。如果缓冲区空了，消费者必须等待生产者生产数据后再继续消费。

2、互斥锁

互斥锁用于保护共享资源，使得在同一时刻只有一个线程可以访问共享资源。

3、pthread 库

Pthreads 库：用于创建和管理线程。pthread_create()：创建线程。pthread_mutex_init()：初始化互斥锁。pthread_mutex_lock()：加锁。pthread_mutex_unlock()：解锁。

三、 实验过程

1、对信号量进程等相关函数、变量进行初始化。

2、编写对 buffer 缓冲区操作（添加、删除）的两个函数。

3、创建生产者和消费者函数。

4、创建主函数进行编译修改运行。

四、 实验结果与分析

```
liao@liao-PC:~/Desktop$ ./main 10 5 5
producer 5 produced 1
10000
producer 1 produced 1
11000
consumer 5 consumed 1
01000
consumer 2 consumed 1
00000
producer 1 produced 2
10000
producer 4 produced 1
11000
consumer 1 consumed 1
01000
producer 2 produced 1
11000
producer 5 produced 2
11100
consumer 1 consumed 2
01100
consumer 4 consumed 1
00100
producer 3 produced 1
10100
consumer 4 consumed 2
00100
producer 1 produced 3
10100
```

五、 问题总结

1、代码编写过程中逻辑或多或少出现混乱，需要加强代码能力。

2、对互斥锁、信号量的初始化要熟练掌握。

3、注意 linux 上运行时的参数传递。

六、 源代码

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
```

```
#include <pthread.h>
#include <semaphore.h>
#include "buffer.h"

buffer_item buffer[BUFFER_SIZE];
// 创建互斥锁和信号量
pthread_mutex_t mutex;
sem_t    full,empty;

void buffer_out(){
        int i;
        for ( i = 0; i <BUFFER_SIZE ; ++i) {
                printf("%d",buffer[i]);
        }
        printf("\n");
}

int insert_item(int a,int b){
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        int i;
        for(i = 0;i < BUFFER_SIZE;i++){
                if(buffer[i] == 0){
                        buffer[i] = 1;//找到空的位置
                        break;
                }
        }

        if(i == BUFFER_SIZE) return -1;
        printf("producer %d produced %d\n",a,b);
```

```
        buffer_out();
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        return 0;
}
int remove_item(int a,int b){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int i;
        for(i = 0;i < BUFFER_SIZE;i++){
                if(buffer[i] != 0){
                        buffer[i] = 0;
                        break;
                }
        }
        if(i == BUFFER_SIZE) return -1;
        printf("consumer %d consumed %d\n",a,b);//第几个吃了几个食物
        buffer_out();
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        return 0;
}

void * producer(void * param){
        int* num = (int*)param;
        int food_id=0;
        while(1){
                sleep(((rand() % 10) + 1));
                if(insert_item(*num,++food_id))
                        printf("report error condition\n");
```

```
        }
}

void * consumer(void *param){
        int* num = (int*)param;
        int food_id=0;
        while(1){
                sleep(((rand() % 10) + 1));
                if(remove_item(*num,++food_id))
                        printf("report error condition\n");
        }
}


int main(int argc,char *argv[])
{
        //输入等待时间、生产者、消费者数量
        if(argc != 4) return -1;
        int sleep_time = atoi(argv[1]);
        int producer_num = atoi(argv[2]);
        int consumer_num = atoi(argv[3]);

        //初始化互斥锁和信号量，以及初始化缓冲区
        pthread_mutex_init(&mutex,NULL);
        sem_init(&empty,0,BUFFER_SIZE);
        sem_init(&full,0,0);
        int i;
        for(i=0;i<BUFFER_SIZE;i++) buffer[i]=0;
```

```
    //创建生产者、消费者线程
    pthread_t producer_tid[producer_num];
    int producer_id[producer_num];
    for(i = 0;i < producer_num;i++){
        producer_id[i] = i + 1;
        pthread_create(&producer_tid[i],NULL,producer,&producer_id[i]);
    }
    pthread_t consumer_tid[consumer_num];
    int consumer_id[consumer_num];
    for(i = 0;i < consumer_num;i++){
        consumer_id[i] = i + 1;
        pthread_create(&consumer_tid[i],NULL,consumer,&consumer_id[i]);
    }

    //休眠
    sleep(sleep_time);
    //退出
    return 0;
}
```